

Track A→Ateko Project: “Mini Commerce Backend”

Scenario

You are building a backend service for a product and ordering platform used by an e-commerce client. The system must expose REST APIs that allow clients to browse products, create orders, and retrieve analytics.

The database required for this assignment is provided to you on Brightspace under the folder:

Week3/Track A Ateko/Ateko Db.sql

Figure 1 shows the EER Diagram of the database we are using in this assignment. Make sure that you import and run the SQL script successfully before starting your implementation.

General Instructions

Make sure that you are following the general instructions of:

- Using FastAPI as your backend framework
- Using SQLAlchemy ORM for database interaction
- Configuring connection pooling in your engine
- Using a .env file for database credentials
- Testing your endpoints using Postman
- Verifying your API documentation in Swagger/OpenAPI
- Your application must run successfully using:

Command prompt

uvicorn main:app --reload

Include a README.md file explaining:

- How to install dependencies
- How to configure the database
- How to run the application

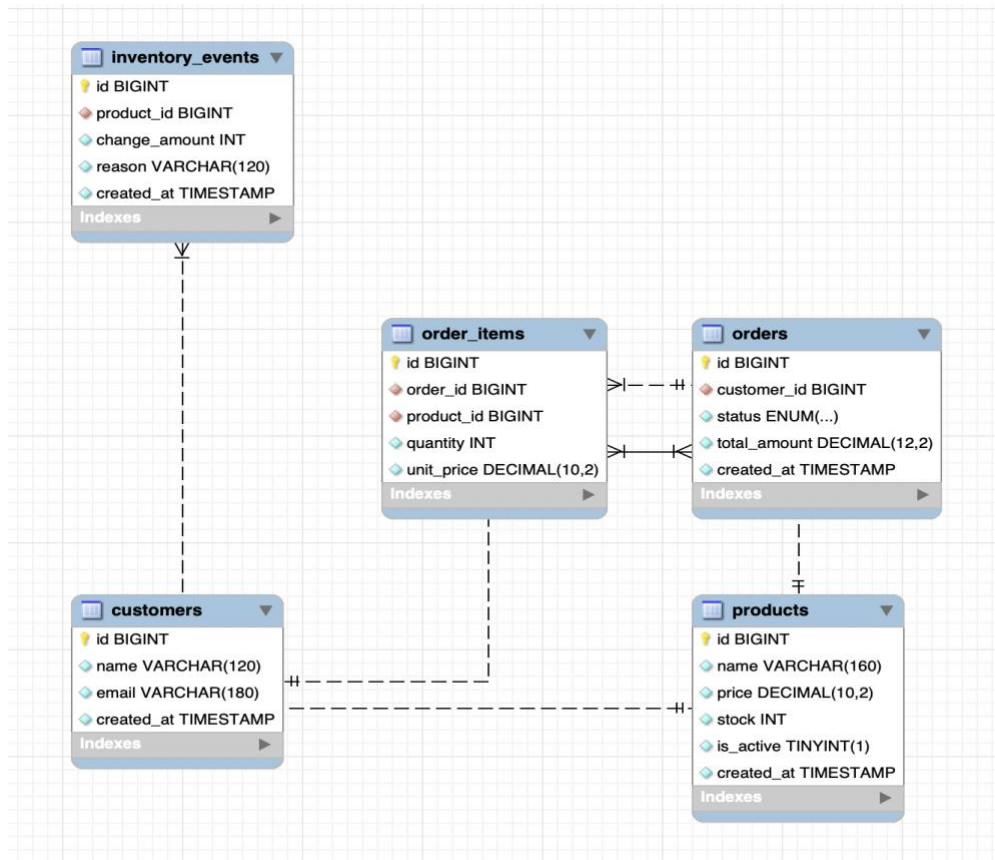


Figure 1. Ateko Database EER Diagram

Part 1 – Product APIs

1. GET /products

What is expected:

You must build a filtering endpoint that allows users to retrieve products based on price range and active status.

This endpoint must:

- Return only active products by default
- Support optional query parameters:
 - min_price
 - max_price
 - active_only

Example:

```
GET /products?min_price=20&max_price=150&active_only=true
```

You should dynamically build the query based on provided parameters.

2. Offset-Based Pagination

Endpoint:

```
GET /products/paged
```

What is expected:

Implement traditional offset-based pagination using `limit` and `offset`.

Example:

```
GET /products/paged?limit=5&offset=10
```

This should return only a subset of products instead of all rows.

3. Cursor-Based Pagination

Endpoint:

```
GET /products/cursor
```

What is expected:

Implement cursor-based pagination using `created_at` and `id` as a stable ordering.

This endpoint must:

- Accept `limit`
- Accept `cursor`
- Return the next page after the cursor

Briefly explain in your README why cursor-based pagination scales better than offset pagination.

Part 2 – Order Creation API

4. POST /orders

What is expected:

You must implement full order creation logic that interacts with multiple tables.

Input:

- `customer_id`
- List of items:
 - `product_id`
 - `quantity`

The endpoint must:

1. Validate customer existence
2. Validate product existence
3. Check stock availability
4. Deduct stock
5. Create order record
6. Create order items
7. Calculate `total_amount`

Important:

Do not implement transaction management yet. This is intentional.

5. GET /orders/{order_id}

What is expected:

Return complete order details including:

- Order information
- Customer information
- List of order items

This should demonstrate proper relationship handling using ORM joins.

Part 3 – Analytics (ORM vs Raw SQL)

Endpoint:

GET /analytics/top-products

Create two versions:

- /analytics/top-products/orm
- /analytics/top-products/sql

What is expected:

Return the top 5 products ranked by total revenue.

Each response must include:

- product_id
- product_name
- total_quantity_sold
- total_revenue

One implementation must use ORM aggregation.

The other must use raw SQL with GROUP BY and SUM.

Reflection (Required)

What is expected:

Write a short technical reflection (max 200 words) answering the following question in the **Part3 Answers.txt** file in your git repository:

- Which implementation was easier?
- Which gave more control?
- When would you prefer raw SQL in production?

Be analytical, not descriptive.

Part 4 – Connection Pooling

What is expected:

Configure your SQLAlchemy engine with:

- `pool_size`
- `max_overflow`
- `pool_pre_ping=True`

In your **Part4_Answers.txt**, answer:

1. What happens if 50 users hit your API simultaneously?
2. What happens if `pool_size = 3`?
3. What could happen if no pooling is configured?

What I will look for

- Clean filtering queries
- Correct pagination behavior
- Correct join and aggregation in analytics